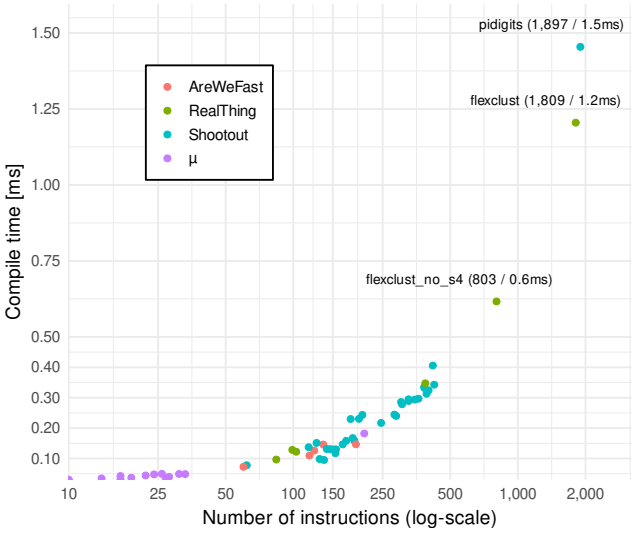



**Figure 2.** Compilation speed.

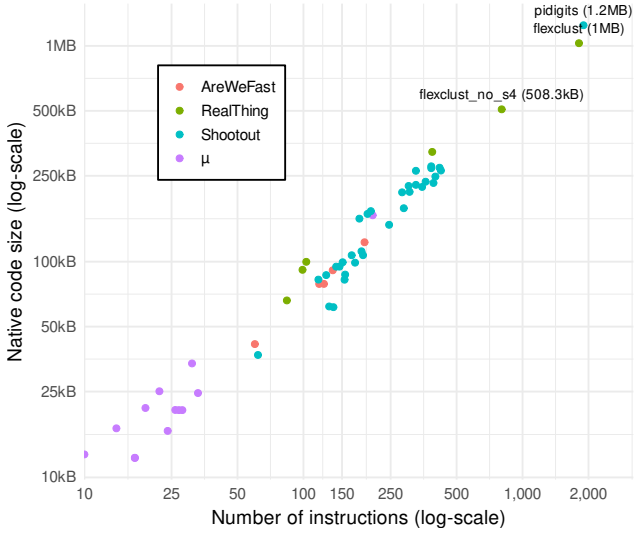



**Figure 3.** Compilation size of the generated native code.

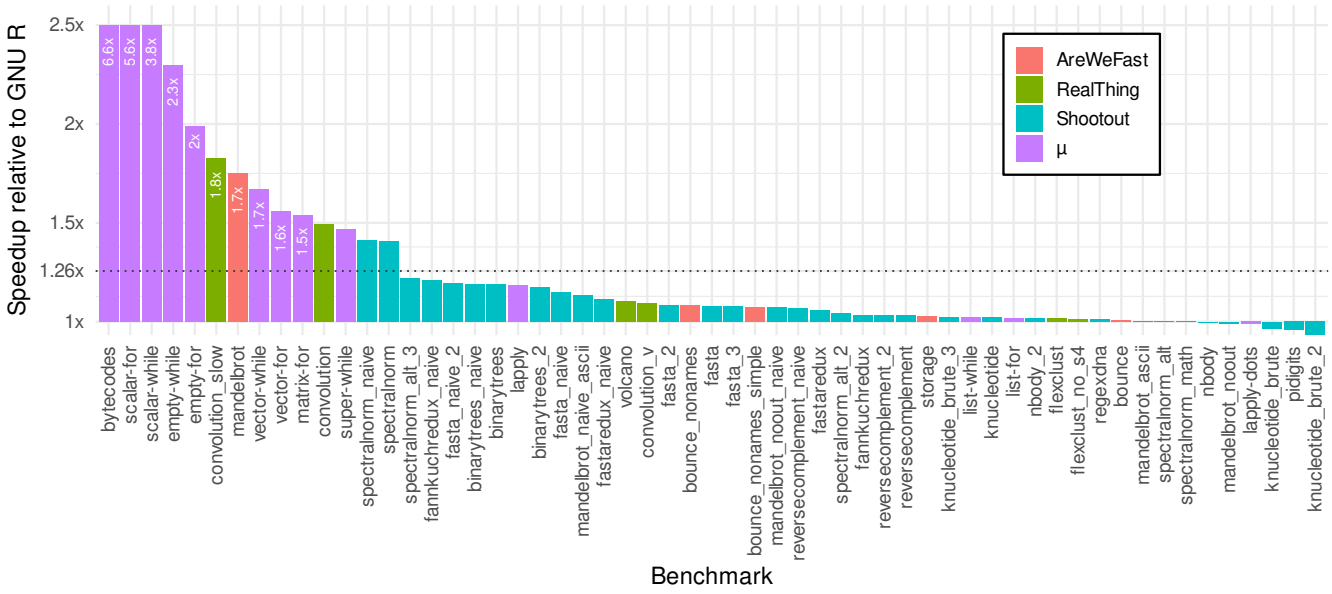



**Figure 4.** The speedup over GNU R. The dotted line indicates the geometric mean.

which we can observe the effect of dispatch and instruction decoding overhead removed.

Lastly, it is important to note that, since we had to adapt the implementation of bytecode instructions for our use case, we do not share the exact same code as the GNU R interpreter, and it is therefore impossible to do a clean performance comparison of just the copy-and-patch technique itself (like it could be done in related projects). This likely means that there exist performance differences that may not come from the copy-and-patch approach, but are instead due to the differences in the implementation of the instructions. Some of the performance benefits shown in this section could therefore have been related to an issue in the interpreter which might miss an unboxing opportunity or similar optimizations.

### 4.4 Performance of Optimizations

From the optimization techniques that we have implemented, the major contributor to the performance improvements is the use of the relative memory model. This model allows for more efficient memory addressing by using 32-bit relative offsets instead of full 64-bit absolute addresses. This